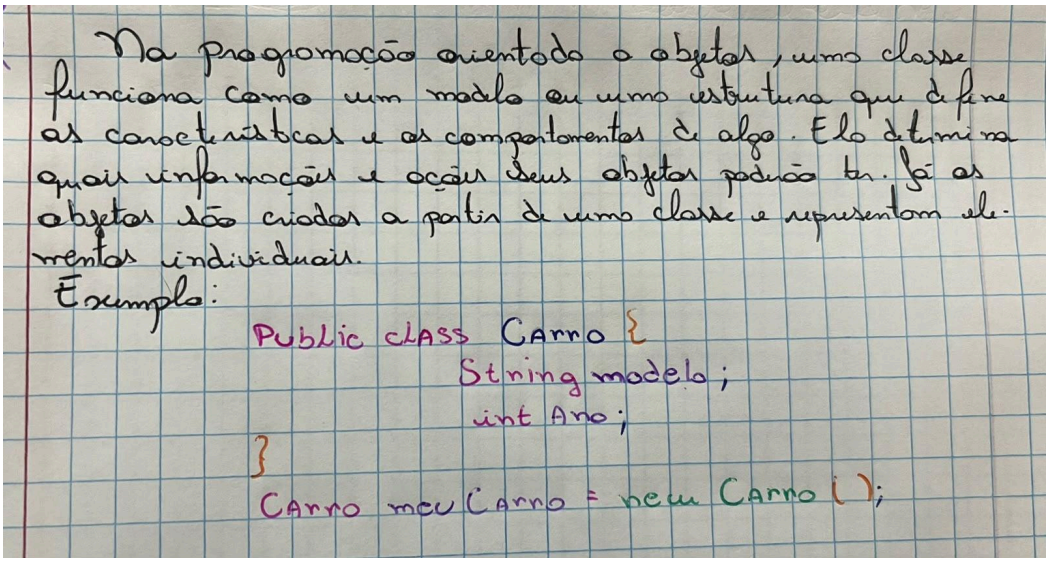
Recuperação 1º Bimestre 2026 - Orientação a Objetos

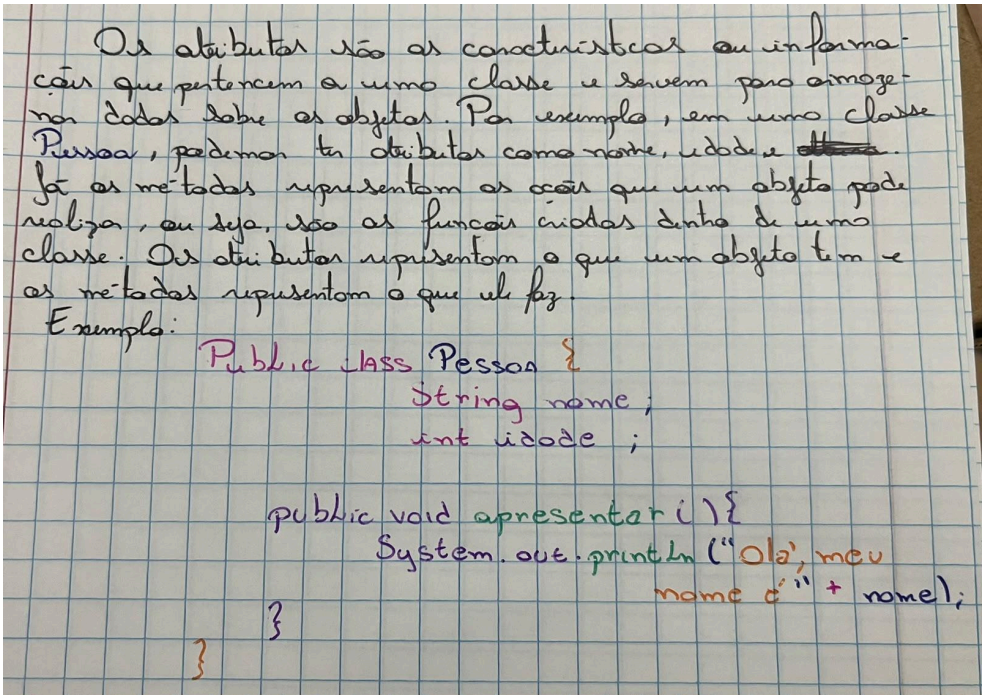
Aluna: Ana Clara Espíndola
Turma: TADS26

Explicando os seguintes conceitos no contexto de orientação a objetos:

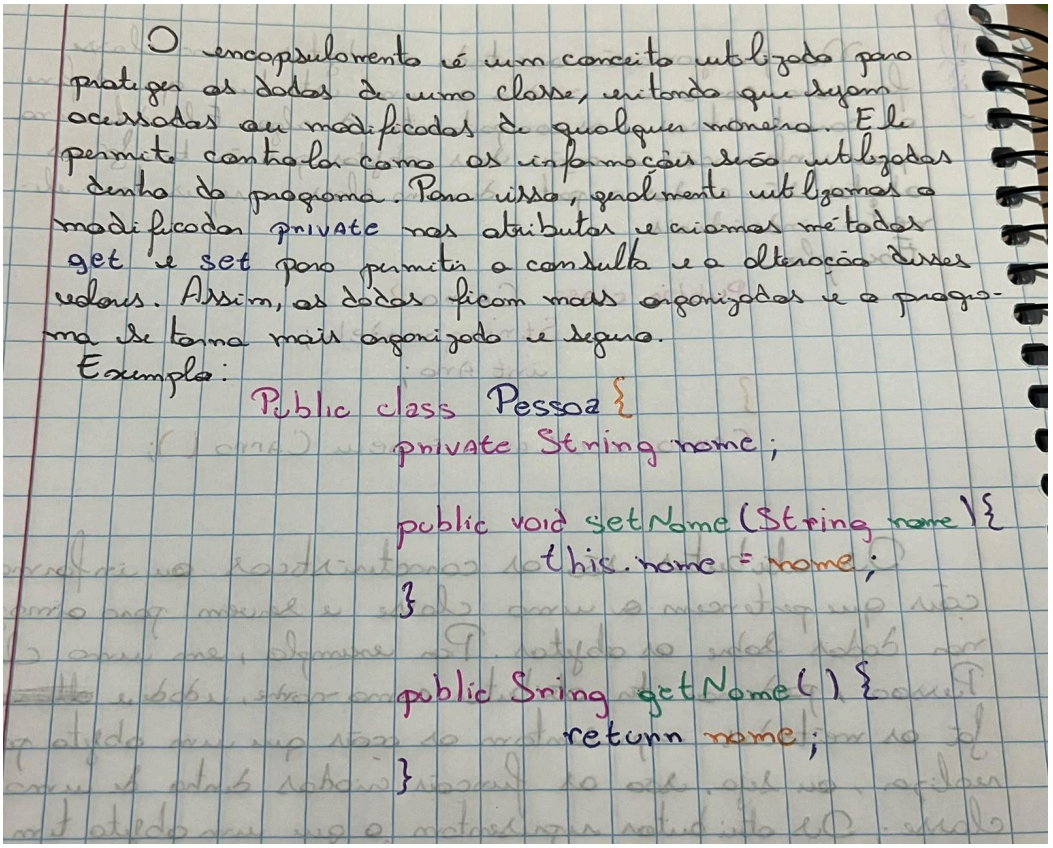
- Classes e Objetos:



- Atributos e Métodos:



- Encapsulamento:



O encapsulamento é um conceito utilizado para proteger os dados de uma classe, evitando que sejam acessados ou modificados de qualquer maneira. Ele permite controlar como as informações serão utilizadas dentro do programa. Para isso, geralmente utilizamos o modificador private nos atributos e criamos métodos get e set para permitir a consulta e a alteração desses valores. Assim, os dados ficam mais organizados e o programa se torna mais organizado e seguro.

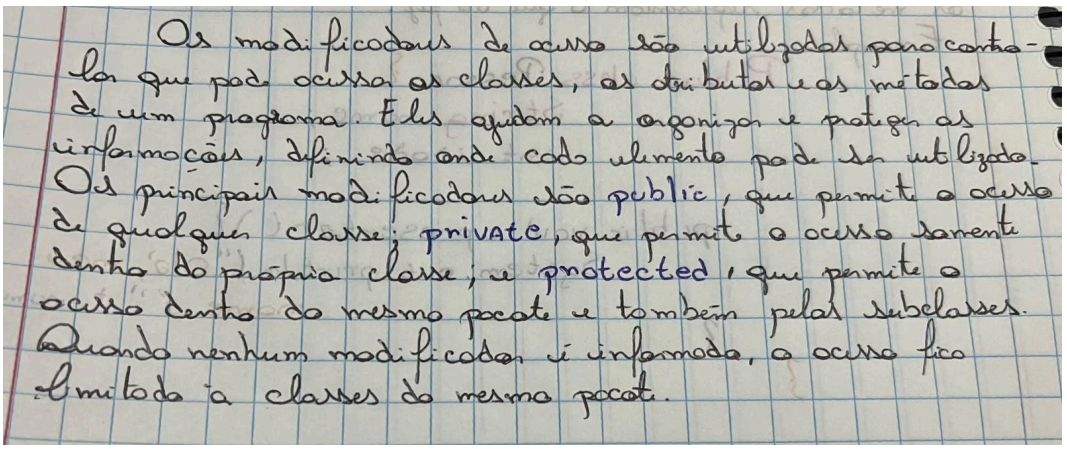
Exemplo:

```
Public class Pessoa {
        private String nome;

        public void setNome(String nome){
                this.nome = nome;
        }

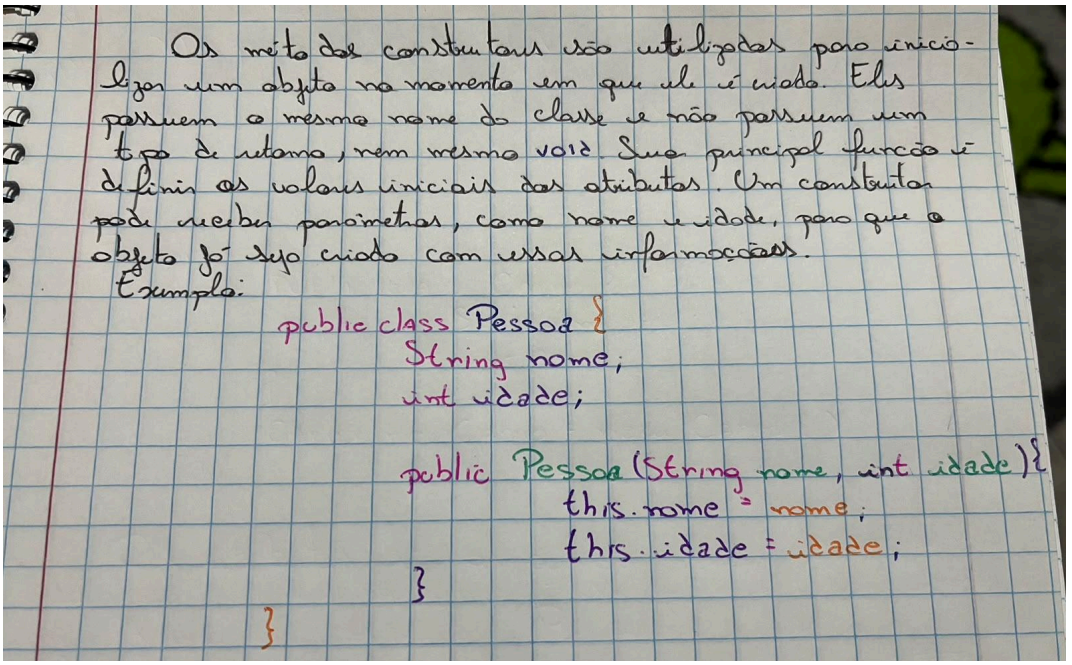
        public Sring getNome() {
                return nome;
        }
```

- Modificadores de Acesso:



Os modificadores de acesso são utilizados para controlar quem pode acessar as classes, os atributos e os métodos de um programa. Eles ajudam a organizar e proteger as informações, definindo onde cada elemento pode ser utilizado. Os principais modificadores são public, que permite o acesso de qualquer classe; private, que permite o acesso somente dentro da própria classe; e protected, que permite o acesso dentro do mesmo pacote e também pelas subclasses. Quando nenhum modificador é informado, o acesso fica limitado a classes do mesmo pacote.

- Métodos Construtores:



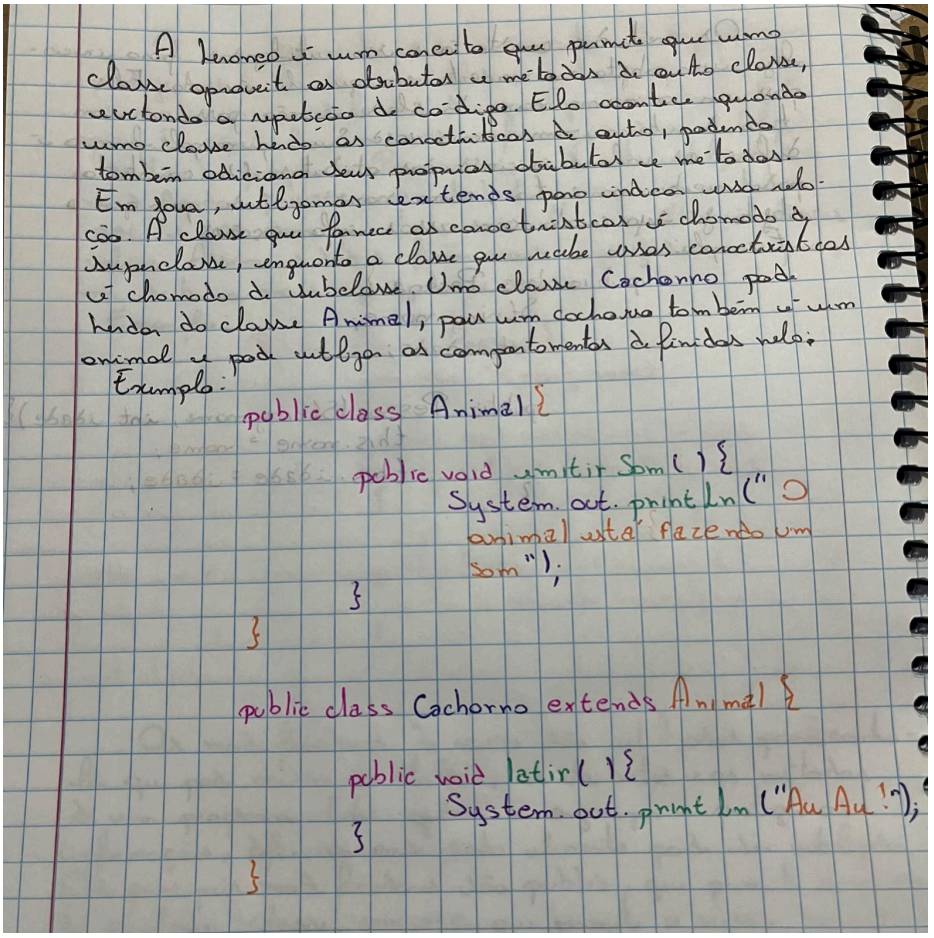
Os métodos construtores são utilizados para iniciá-lizar um objeto no momento em que ele é criado. Eles possuem o mesmo nome da classe e não possuem um tipo de retorno, nem mesmo void. Sua principal função é definir os valores iniciais dos atributos. Um construtor pode receber parâmetros, como nome e idade, para que o objeto já seja criado com essas informações.

Exemplo:

```
public class Pessoa {
    String nome;
    int idade;

    public Pessoa(String nome, int idade){
        this.nome = nome;
        this.idade = idade;
    }
}
```

- Herança:



A herança é um conceito que permite que uma classe aproveite os atributos e métodos de outra classe, evitando a repetição de código. Ela acontece quando uma classe herda as características de outra, podendo também adicionar seus próprios atributos e métodos. Em Java, utilizamos extends para indicar essa relação. A classe que fornece as características é chamada de superclasse, enquanto a classe que recebe essas características é chamada de subclasse. Uma classe Cachorro pode herdar da classe Animal, pois um cachorro também é um animal e pode subfazer os comportamentos definidos nela.

Exemplo:

```
public class Animal {

    public void emitirSom() {
        System.out.println("O animal está fazendo um som");
    }
}

public class Cachorro extends Animal {

    public void latir() {
        System.out.println("Au Au!");
    }
}
```